

SHPATH

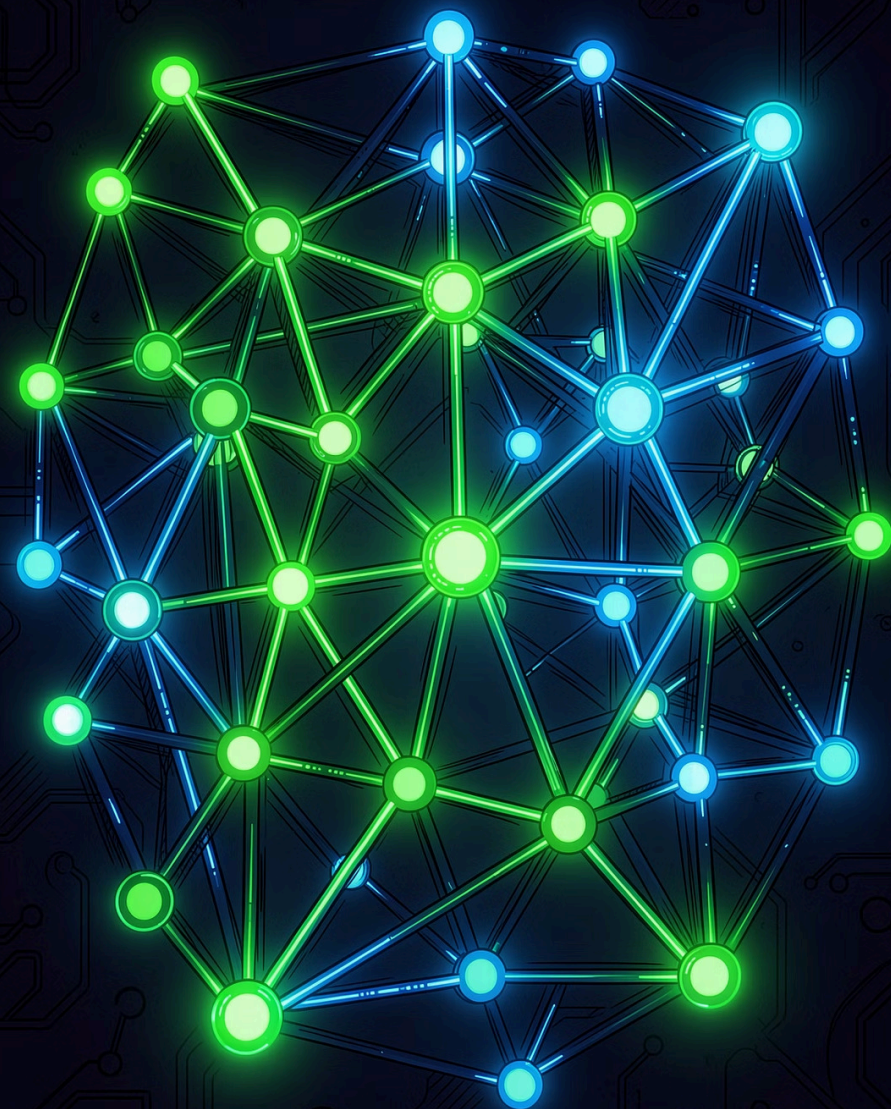
The Shortest Path

GRUPO C

SITE DA SPOJ

FEITO EM PYTHON 3

Lucas Barreira · José Eugênio · Carlos Huan · Mateus Rocha



Made with GAMMA

O Problema



Cidades = Vértices

Cada cidade é um nó



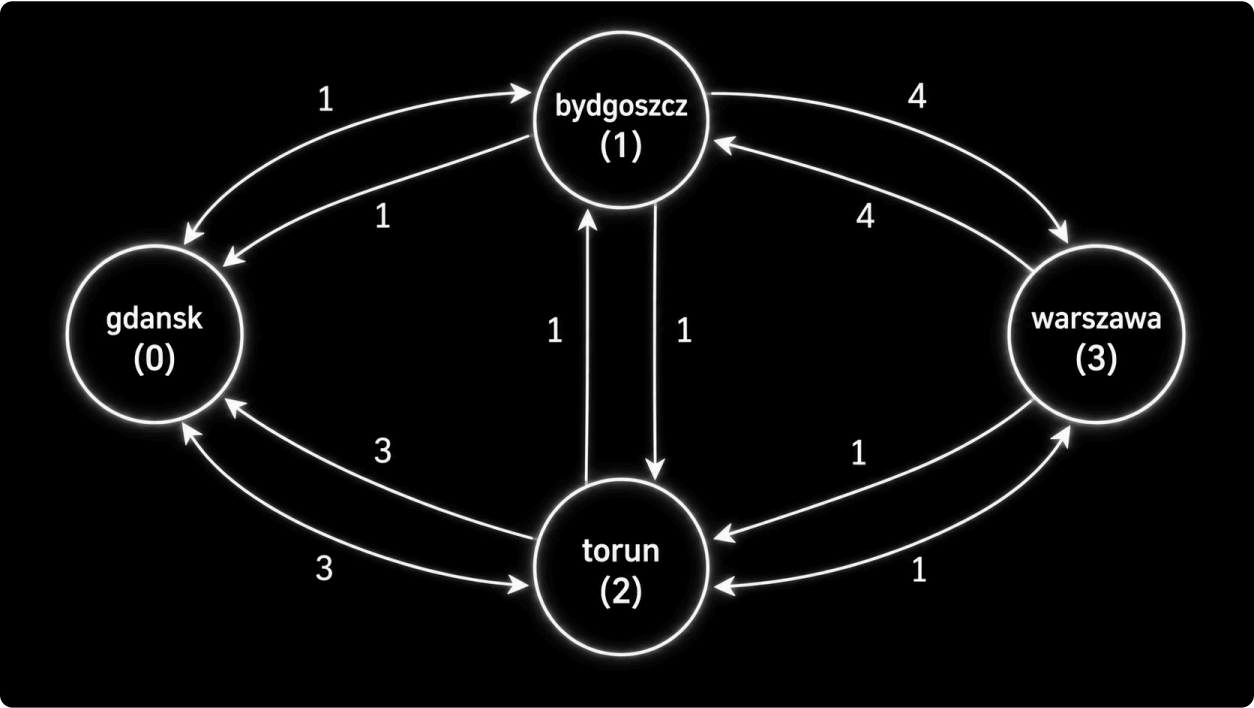
Conexões = Aresta

Cada rota tem um custo



Consulta = Custo

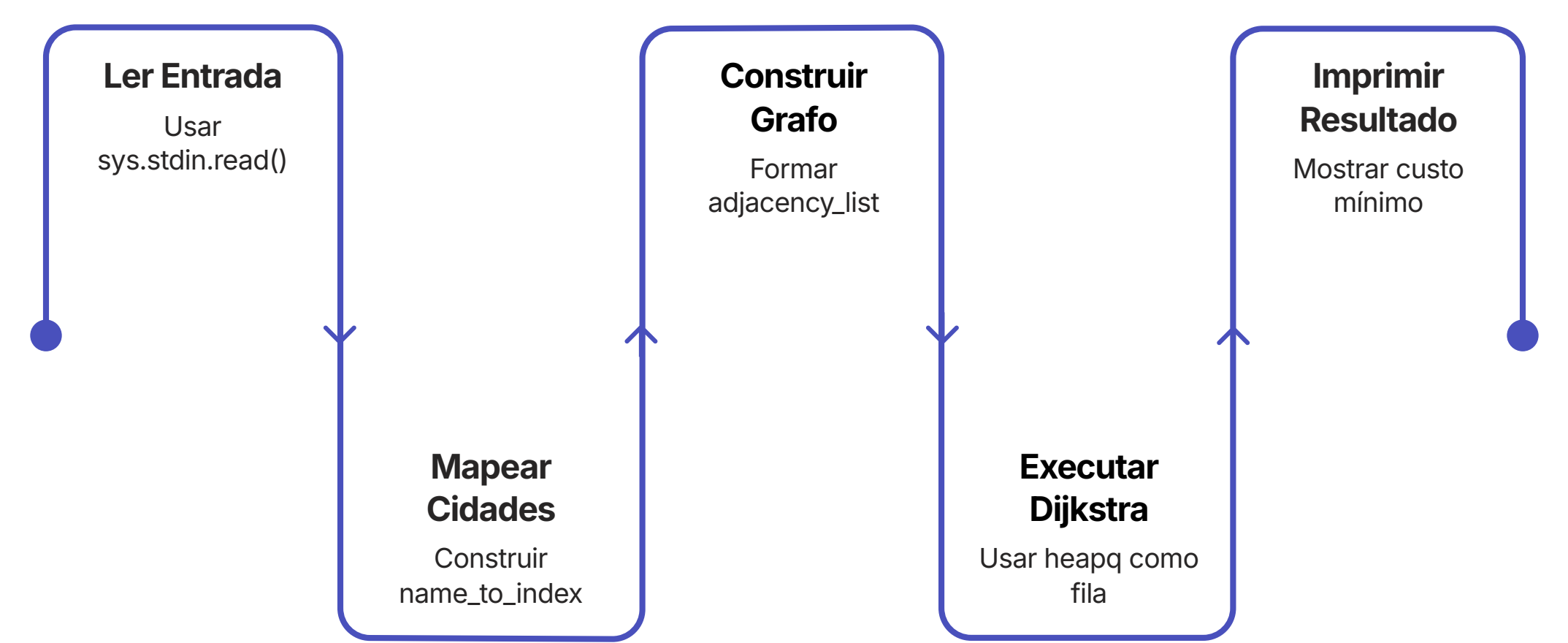
Menor custo de A até B



Cidades do Exemplo

gdansk	0
bydgoszcz	1
torun	2
warszawa	3

Execução do Programa



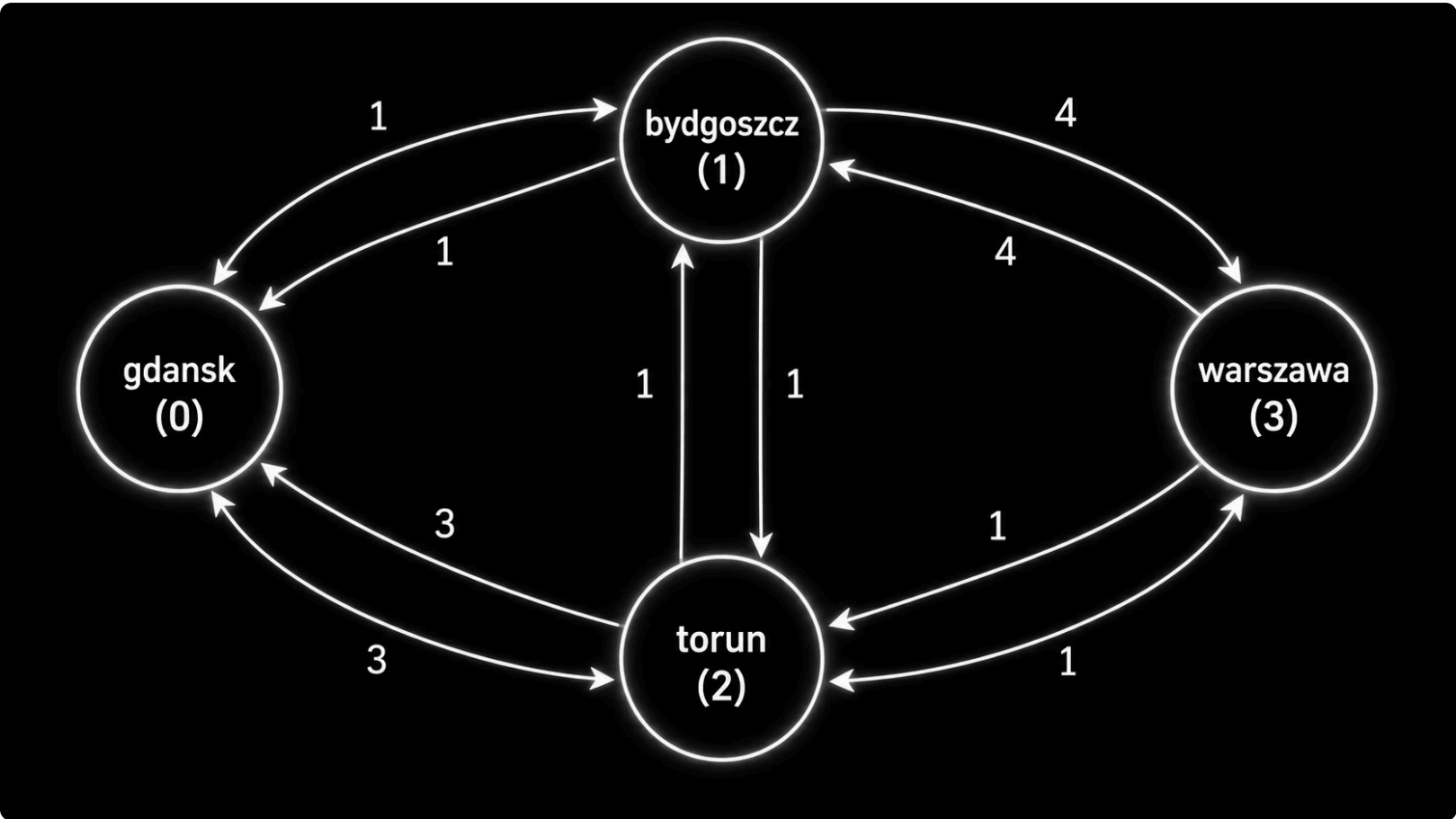
Fluxo completo da solução: da leitura da entrada até a impressão do menor custo.



Como o Dijkstra encontra o menor caminho?

Sempre escolhe o vértice com **menor custo acumulado** — garantido quando todos os custos são positivos.

01	02	03
Origem	Heap	Menor Custo
Start from source	Use priority queue	Pick minimum cost vertex
04		
Relaxar Vizinhos		
Relax neighbor edges		



Algoritmo: Dijkstra com heapq

Passos do Algoritmo

01

Inicializar distâncias como INF e a origem como 0

02

Inserir a origem na fila de prioridade mínima (heapq)

03

Retirar o vértice com menor custo acumulado

04

Relaxar as arestas: se $\text{dist}[u] + \text{custo} < \text{dist}[v]$, atualizar

05

Repetir até a fila esvaziar ou o destino ser confirmado

06

Otimização: parada antecipada ao atingir o destino

Núcleo do Dijkstra

```
def dijkstra(graph, source, target, distance, mark, run_id):
    distance[source] = 0
    heap = [(0, source)]
    while heap:
        cost, city = heappop(heap)
        # ignora entradas desatualizadas
        if mark[city] != run_id or cost != distance[city]:
            continue
        if city == target:
            return cost
        for neighbor, edge_cost in adj:
            new_cost = cost + edge_cost
            if new_cost < distance[neighbor]:
                distance[neighbor] = new_cost
                heappush(heap, (new_cost, neighbor))
```

Otimizações e Decisões de Implementação

Otimizações Principais

1. run_id (marca de validade)

Em vez de zerar o vetor distance a cada Dijkstra ($O(V)$), usamos um contador run_id

Se `mark[v] != run_id`, o vértice está "não visitado" nesta rodada

Evita $O(V)$ de reset por consulta

2. Lista de adjacência achatada

Adjacência armazenada como lista `[viz0, custo0, viz1, custo1, ...]`

Acesso por índice inteiro evita overhead de objetos Python

Melhora cache locality

3. Leitura em bloco (`sys.stdin.buffer`)

Toda entrada é lida de uma vez com `sys.stdin.buffer.read().split()`

Elimina o custo de múltiplas chamadas de `input()` em Python

Crucial para grandes entradas

Por que Dijkstra e não Bellman-Ford?

Parada antecipada

Quando o nó retirado da fila é exatamente o destino consultado, o Dijkstra encerra imediatamente

Não precisa encontrar todos os menores caminhos

Todos os custos são positivos

Dijkstra garante que, ao confirmar a distância de um vértice (pop da fila), ela é ótima

Bellman-Ford seria $O(V \cdot E)$ por consulta — inviável para 10.000 cidades

Complexidade e Conclusão

$O((V+E) \log V)$

 Tempo

Por consulta com heapq

$O(V+E)$

 Memória

Grafo + distâncias

1

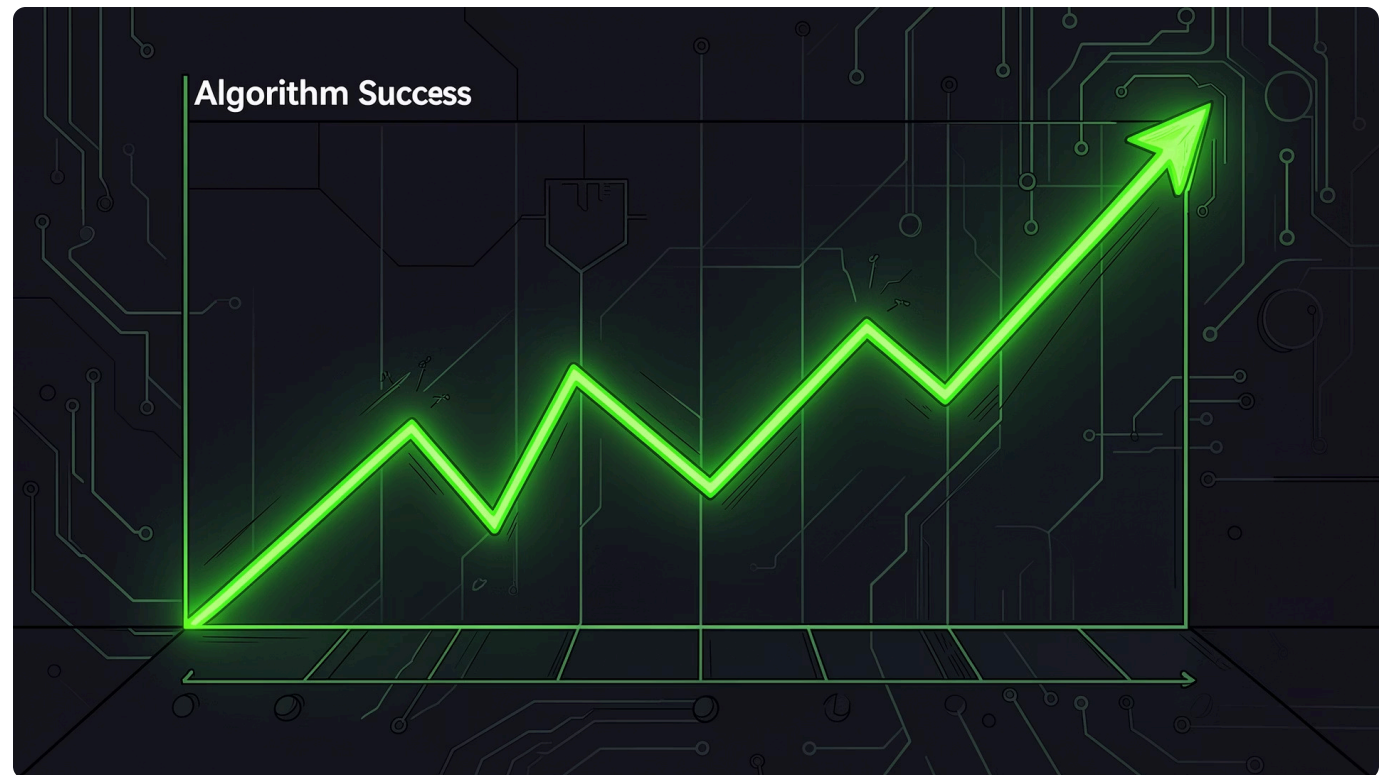
 Otimização

Parada antecipada no destino

Conclusão

Dijkstra + heapq resolve o **SHPATH** de forma eficiente para grafos com **pesos positivos**.

- Grafo direcionado e ponderado
- Fila de prioridade mínima
- Relaxamento guloso de arestas



SPOJ SHPATH

GRUPO C

PYTHON 3